

Dart

Introduction Lab

Mobile Application Development Course

```
void main() {  
    final message = 'Hello, Dart!';  
    final numbers = [1, 2, 3, 4, 5];  
    for (var n in numbers) {  
        print(n * 2);  
    }  
}
```



Teaching Assistant

Abd El-Rahman Mostafa

- abdelrahman.mostafa.aast@gmail.com

```
void main() {  
    final message = 'Hello, Dart!';  
    final numbers = [1, 2, 3, 4, 5];  
    for (var n in numbers) {  
        print(n * 2);  
    }  
}
```



Dart Setup & Your First Program

- program's entry from **main()** like C++, and C
- Use **print()** to show output
- Try “**Hello, Dart!**” and check your SDK version

N.B.: DartPad is an online alternative

A laptop is shown in the foreground, slightly out of focus, with its screen displaying Dart code. The code is written in a light blue font on a dark background. The code includes an import statement for 'dart:io', a main function definition, and two print statements: one for a string 'Hello, Dart!' and another for Platform.version.

```
import 'dart:io';

void main() {
    print('Hello, Dart!');
    print(Platform.version);
}
```

Comments & Variables

COMMENTS

Use single-line using `//` and multi-line comments using `/* */`

CORE TYPES

num, int, double, String, bool

DECLARE VARIABLES

Use var for type inference or specify the type explicitly

VAR & DYNAMIC & CONST

var adapts once, dynamic can change, const stays fixed at compile time

```
// This is a single-line comment
/*
  This is a multi-line comment
  It can span multiple lines
*/

int count = 42;
double price = 19.99;
String name = 'Dart';
bool isActive = true;

var inferred = 'hello';
dynamic anything = 123;
const pi = 3.14159;
```

Operators in Dart

Unary operators
Binary operators
Ternary operators
Comparison operators
Logical operators

Arithmetic

+ - * / ~/ %

Relational

== != <= >= < >

Logical

&& || !

[Check for all supported operators](#)



Input and Output Practice

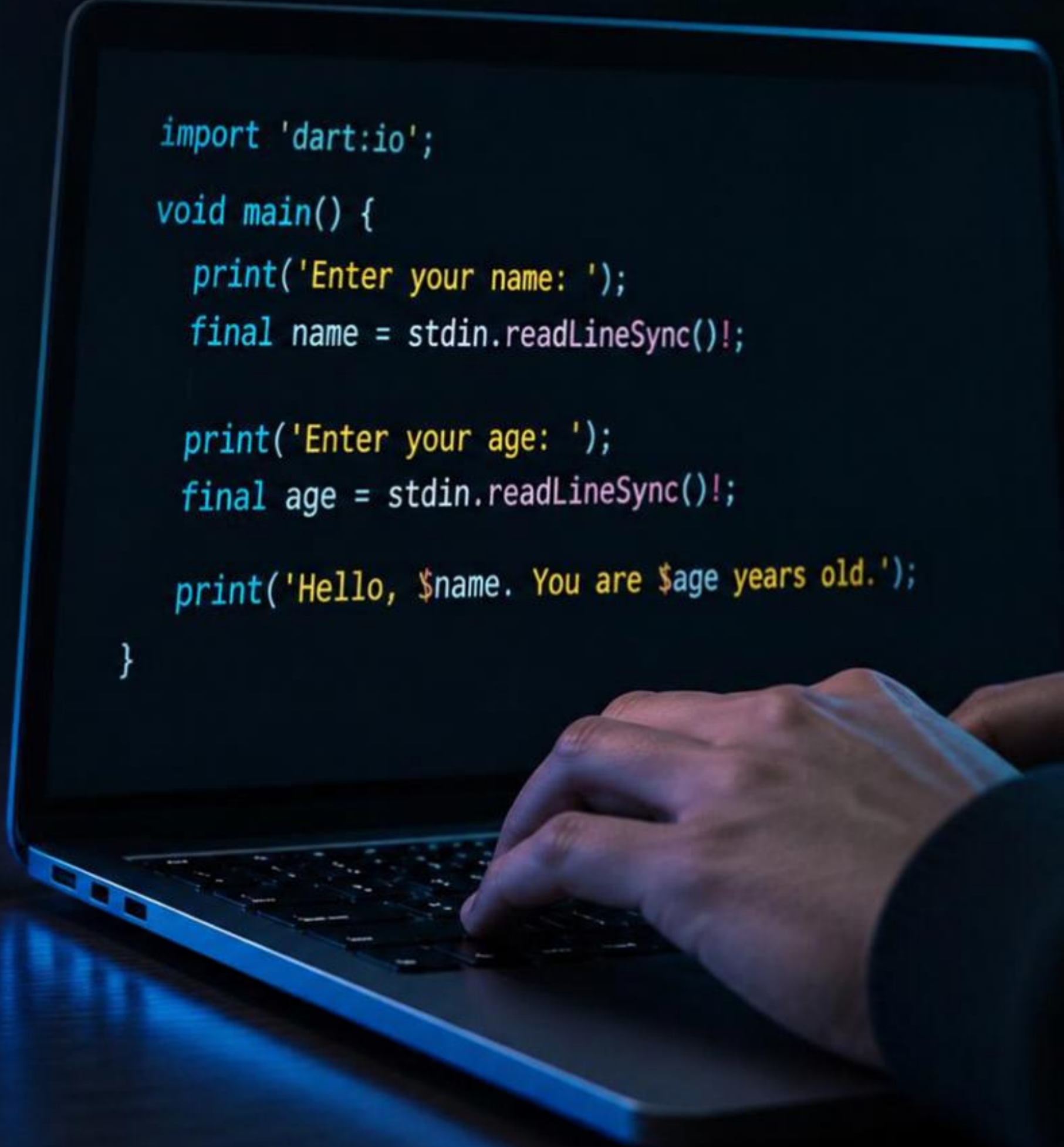
- Read user input with `stdin.readLineSync()`
- It returns `String?` (nullable)
- Use `!` to assert a non-null value (like TypeScript)

```
import 'dart:io';

void main() {
  print('Enter your name: ');
  final name = stdin.readLineSync()!;

  print('Enter your age: ');
  final age = stdin.readLineSync()!;

  print('Hello, $name. You are $age years old.');
```



```
}
```

Numbers & Strings Essentials

Round doubles with `.round()`

Use `%` to check even or odd

Work with text using `.split()`; `.join()`; and `.reversed`

Remember: strings in Dart are `immutable`

Collections

List basics in Dart

Create ordered lists with `List<int>`

Use `.add()` , `.remove()` and `.sort()` to update and organize items

Use `.reduce()`; to find max and min values

Remember: **order matters** and indexing starts at **zero**



```
list.add(value)
```

```
list.remove(value)
```

```
list.sort()
```

```
list.reduce((a, b) => ...)
```

Collections

Set & Map Overview

- **Sets** hold unique, unordered values

Use add, remove, contains, union, and intersection to manage and combine sets

- **Maps** store key-value pairs

Declare, print, check keys, and look up values safely

Setting Flutter with Dart

[Check that markdown](#)